

Dokumentacja Techniczna Projektu: Vehicle Registry

Autor: Maciej Zawada, kl. 3e

Niniejsza dokumentacja opisuje strukturę oraz poszczególne elementy (klasy i metody) aplikacji konsolowej służącej do rejestracji i zarządzania różnymi typami pojazdów (lądowymi, wodnymi i powietrznymi).

1. Struktura Projektu

Projekt przyjmuje następującą strukturę katalogów i plików:

```
src/
├── zse/
│   └── en/
│       ├── Main.java
│       ├── Owner.java
│       ├── Vehicle.java
│       ├── LandVehicle.java
│       ├── WaterVehicle.java
│       ├── AirVehicle.java
│       └── VehicleRegistry.java
```

2. Opis Klas i Metod

2.1. Klasa `Main`

Główna klasa aplikacji zawierająca punkt wejścia do programu. Odpowiada za interfejs użytkownika w konsoli, pobieranie danych wejściowych oraz interakcję z rejestrem pojazdów.

- Metody:**

- `public static void main(String[] args)`
 - Główna pętla programu. Wyświetla menu, pobiera od użytkownika wybór akcji (dodanie pojazdu, wylistowanie, serwisowanie, wyjście). Zbiera i waliduje dane wejściowe (`m.in.` wymusza 17-znakowy numer VIN, obsługuje daty rejestracji), tworzy obiekty odpowiednich klas dziedziczących po `Vehicle` i dodaje je do obiektu `VehicleRegistry`. Przechwytuje również błędy formatowania danych (`try-catch`).

2.2. Klasa `Owner`

Reprezentuje właściciela pojazdu. Klasa przechowuje podstawowe dane osobowe.

- **Pola:**

- `private final String firstName` – Imię właściciela.
- `private final String lastName` – Nazwisko właściciela.

- **Konstruktory:**

- `public Owner(String firstName, String lastName)`
 - Inicjalizuje obiekt właściciela z podanym imieniem i nazwiskiem.

- **Metody:**

- `public String toString()`
 - Przesłania standardową metodę `toString()`. Zwraca połączone imię i nazwisko właściciela w formie pojedynczego łańcucha znaków (np. "Jan Kowalski").
-

2.3. Klasa abstrakcyjna `Vehicle`

Bazowa klasa abstrakcyjna dla wszystkich typów pojazdów w systemie. Definiuje wspólne cechy i wymusza implementację specyficznych zachowań w klasach pochodnych.

- **Pola:**

- `private final String vin` – Numer identyfikacyjny pojazdu (VIN).
- `private final String brand` – Marka pojazdu.
- `private final Owner owner` – Obiekt reprezentujący właściciela pojazdu.
- `private final LocalDate registrationDate` – Data rejestracji pojazdu.

- **Konstruktory:**

- `public Vehicle(String vin, String brand, Owner owner, LocalDate registrationDate)`
 - Inicjalizuje podstawowe właściwości współdzielone przez wszystkie pojazdy.

- **Metody:**

- `public LocalDate getRegistrationDate()`
 - Zwraca datę rejestracji pojazdu.
- `protected void printBaseInfo()`
 - Wypisuje w konsoli sformatowane, podstawowe informacje o pojeździe (data rejestracji, VIN, marka, dane właściciela).
- `public abstract void displayDetails()`
 - Metoda abstrakcyjna. Klasy dziedziczące muszą dostarczyć własną implementację wyświetlającą szczegóły specyficzne dla danego typu pojazdu.
- `public abstract void serviceVehicle()`
 - Metoda abstrakcyjna. Klasy dziedziczące muszą dostarczyć własną implementację symulującą proces serwisowania pojazdu.

2.4. Klasa `LandVehicle`

Klasa reprezentująca pojazd lądowy. Dziedziczy po klasie `Vehicle`.

- **Pola:**

- `private final int odometer` – Stan licznika przebiegu w kilometrach.

- **Konstruktory:**

- `public LandVehicle(String vin, String brand, Owner owner, int odometer, LocalDate regDate)`
 - Wywołuje konstruktor klasy bazowej i inicjalizuje pole `odometer`.

- **Metody:**

- `public void displayDetails()`
 - Wywołuje `printBaseInfo()` z klasy bazowej, a następnie dopisuje informacje specyficzne dla pojazdu lądowego (typ: LAND, przebieg w km).
 - `public void serviceVehicle()`
 - Wypisuje w konsoli humorystyczny komunikat związany z serwisowaniem pojazdu lądowego ("Service: Sir, this will work out, sir").
-

2.5. Klasa `WaterVehicle`

Klasa reprezentująca pojazd wodny. Dziedziczy po klasie `Vehicle`.

- **Pola:**

- `private final double displacement` – Wyporność statku w tonach (t).
- `private final boolean submersible` – Flaga określająca, czy pojazd jest zdolny do zanurzenia (np. łódź podwodna).

- **Konstruktory:**

- `public WaterVehicle(String vin, String brand, Owner owner, double displacement, boolean submersible, LocalDate regDate)`
 - Wywołuje konstruktor klasy bazowej oraz inicjalizuje pola `displacement` i `submersible`.

- **Metody:**

- `public void displayDetails()`
 - Wywołuje `printBaseInfo()`, po czym dołącza informacje o typie (WATER), wyporności oraz zdolności do zanurzenia (YES/NO).
 - `public void serviceVehicle()`
 - Wypisuje komunikat związany z serwisowaniem pojazdu wodnego ("Service: Replacing the rear drive propeller (XDDDD)").
-

2.6. Klasa `AirVehicle`

Klasa reprezentująca pojazd powietrzny. Dziedziczy po klasie `Vehicle`.

- **Pola:**

- `private final int flightHours` – Liczba wylatanych godzin.

- **Konstruktory:**

- `public AirVehicle(String vin, String brand, Owner owner, int flightHours, LocalDate regDate)`
 - Wywołuje konstruktor klasy bazowej i inicjalizuje pole `flightHours`.

- **Metody:**

- `public void displayDetails()`
 - Wywołuje `printBaseInfo()`, a następnie wyświetla informacje specyficzne (typ: AIR, godziny lotu).
- `public void serviceVehicle()`
 - Wypisuje komunikat serwisowy dla pojazdu powietrznego ("Service: he will fly like lightning").

2.7. Klasa `VehicleRegistry`

Klasa zarządza kolekcją zarejestrowanych pojazdów, wykorzystując dynamicznie rozszerzaną tablicę wewnętrzną, co pozwala na przechowywanie dowolnej liczby obiektów bez użycia gotowych list (np. `ArrayList`).

- **Pola:**

- `private Vehicle[] storage` - Wewnętrzna tablica służąca do przechowywania referencji do obiektów pojazdów (dziedziczących po klasie abstrakcyjnej `Vehicle`).
- `private int size` - Licznik przechowujący aktualną liczbę zarejestrowanych pojazdów (oraz wskazujący na pierwszy wolny indeks w tablicy).

- **Konstruktory:**

- `public VehicleRegistry()`
 - Bezparametrowy konstruktor inicjalizujący pusty rejestr. Alokuje pamięć dla początkowej tablicy `storage` o rozmiarze 2 elementów i ustawia początkową wartość `size` na 0.

- **Metody:**

- `public void register(Vehicle v)`
 - Rejestruje nowy pojazd w systemie. Przed dodaniem elementu sprawdza, czy tablica `storage` nie jest pełna. Jeśli tak, wywołuje metodę `expandCapacity()`. Następnie dodaje pojazd na końcu (pod indeksem `size`) i inkrementuje licznik.
- `private void expandCapacity()`
 - Prywatna metoda pomocnicza realizująca dynamiczne powiększanie tablicy. Tworzy nową tablicę o dwukrotnie większym rozmiarze (`storage.length * 2`), kopiuje do niej dotychczasowe elementy za pomocą wydajnej metody `System.arraycopy`, a następnie zastępuje starą referencję `storage` nową tablicą.
- `public void listAll()`
 - Wyświetla listę wszystkich zarejestrowanych pojazdów. Jeśli rejestr jest pusty, informuje o tym użytkownika komunikatem "Registry is empty.". W przeciwnym razie iteruje przez wszystkie dodane elementy (do indeksu `size`) i wywołuje na nich polimorficzną metodę `displayDetails()`.
- `public void processMaintenance()`
 - Przeprowadza masowy proces serwisowy. Przechodzi w pętli przez wszystkie dodane do rejestru pojazdy i wywołuje dla każdego z nich odpowiednią dla danego typu implementację metody `serviceVehicle()`.